

Designing a Recommender System for Board Games: Reproducibility Study

Tony Zhang, Prajwal Umesha, Shuo Jiang

University of Minnesota, Minneapolis, MN, USA

zhan6696@umn.edu, umesh006@umn.edu, jian1105@umn.edu

Abstract

We present a reproducibility study of Ion et al. [1], which proposes and evaluates a suite of recommender systems for board games using data from BoardGameGeek (BGG). The original work benchmarks collaborative filtering approaches spanning memory-based neighborhood methods, matrix factorization, and autoencoders against content-based and hybrid methods, across 15 evaluation metrics covering ranking accuracy, diversity, and novelty. As the authors released neither code nor data, we independently re-implement all algorithms and evaluate them on a dataset constructed from a publicly available Kaggle snapshot of BGG [2], comprising 8,081,054 ratings across 21,919 games from 20,310 users after filtering. Our replication confirms the paper's central findings: collaborative filtering methods substantially outperform content-based approaches on ranking accuracy, autoencoders achieve the strongest overall performance, and the popularity baseline is a surprisingly competitive benchmark. Content-based methods, however, produce recommendations that are more novel and diverse. The hybrid CF-CB approach does not offer meaningful gains over the autoencoder alone. Beyond replication, we extend the original study with systematic hyperparameter tuning of autoencoder variants, item-based autoencoder formulations, a Collaborative Denoising Autoencoder (CDAE), Gaussian-noise denoising, and a Graph Neural Network (GNN) recommender. Our results are broadly consistent with the original conclusions, though quantitative differences highlight the sensitivity of recommender system evaluations to dataset composition and the importance of artifact sharing in reproducibility research.

Keywords: recommender systems, board games, reproducibility, collaborative filtering, autoencoders, content-based filtering, BoardGameGeek

1. Introduction

The board game hobby has experienced remarkable growth over the past decade, with new releases increasing by approximately 25% annually and the global market valued at around \$1.5 billion [3]. This proliferation of titles presents a genuine discovery challenge: faced with thousands of options, users often struggle to identify games that align with their tastes, a phenomenon well-studied in the broader consumer choice literature as "decision paralysis" [4]. BoardGameGeek (BGG), the dominant online platform for the hobby, has accumulated tens of millions of user ratings alongside rich structured metadata including game mechanics, themes, categories, playing time, and designer credits, making it an unusually well-resourced testbed for recommender systems research.

Ion et al. [1] exploit this resource to conduct a systematic comparison of recommender approaches in the board game domain. Their study spans collaborative filtering (CF) methods including memory-based user and item neighborhood models, matrix factorization, and three autoencoder architectures, alongside content-based (CB) recommenders and a neural CF-CB hybrid. Evaluated across ranking accuracy, novelty, and diversity metrics, the study concludes that autoencoder-based CF achieves the strongest ranking performance, while simpler CB approaches yield recommendations that are more diverse and novel. The hybrid model offers a middle ground but does not improve on the autoencoder for accuracy.

Reproducing results from published recommender systems papers is notoriously difficult. Evaluation outcomes are sensitive to dataset construction, preprocessing choices, hyperparameter settings, and metric implementations, details that are frequently underspecified in publications. The work of Ion et al. [1] is representative of this challenge: no code was released, the dataset was assembled via a scraper that is no longer available, and several algorithmic details are left ambiguous in the paper.

In this work, we attempt a faithful replication of Ion et al. [1], making the following contributions:

- A complete, independent re-implementation of all algorithms described in the original paper: a popularity baseline, User-User and Item-Item neighborhood-based collaborative filtering, Matrix Factorization, Classic, Denoising, and Variational Autoencoders, content-based kNN and IDF recommenders, and a neural CF-CB hybrid.
- Evaluation on a BGG dataset sourced from Kaggle [2], assessed using all 15 metrics reported in the original paper, specifically Precision@k, Recall@k, NDCG@k, MAP, Diversity, and Novelty for k in {5, 10, 20, 100}, with full documentation of our metric implementations and the assumptions required to resolve ambiguities in the original descriptions.
- A set of extensions that go beyond the original paper, including systematic hyperparameter tuning of autoencoder architectures, item-based variants of all autoencoder models, a Collaborative Denoising Autoencoder (CDAE), a Gaussian-noise denoising strategy, and a Graph Neural Network (GNN) recommender.
- A detailed analysis of the sources of divergence between our results and those of the original paper, distinguishing between differences attributable to dataset mismatch and those arising from underspecified implementation details.

Our results broadly support the qualitative conclusions of Ion et al. [1]: autoencoders lead on ranking accuracy, content-based methods are more diverse and novel, and the popularity baseline is difficult to beat with neighborhood-based approaches. Quantitative differences between our numbers and the original table are present and expected given the dataset mismatch, but the relative ordering of methods is largely preserved. One significant difference that we noted was that the hybrid autoencoder performed much more like the content-based algorithms than the classic user based autoencoders. Our extended experiments further show that hyperparameter tuning can substantially improve autoencoder performance, and that the GNN recommender produces the best results.

The remainder of this paper is structured as follows. Section 2 reviews related work. Section 3 describes our dataset and preprocessing. Section 4 presents the algorithms we implement. Section 5 details our evaluation methodology and reports results. Section 6 describes and evaluates our extensions. Section 7 discusses the implications of our findings. Section 8 concludes.

2. Related Work

Recommender systems have been studied extensively across a wide range of domains. Collaborative filtering methods, which rely solely on user-item interaction data, have historically dominated the field. Memory-based approaches such as User-User and Item-Item filtering [5, 6] compute similarity between users or items directly from rating data and generate recommendations based on the preferences of nearest neighbors. While intuitive, these methods scale poorly to large datasets and struggle with sparse interaction matrices.

Matrix factorization [7] addresses these limitations by decomposing the user-item rating matrix into low-dimensional latent factor representations for both users and items. This allows generalization to unrated items and has shown strong empirical performance across many benchmark datasets. AutoRec [8] extends this line of work by applying autoencoders to collaborative filtering, demonstrating that the

non-linear transformations learned by neural networks can capture latent structure in rating data more effectively than linear factorization. Variational autoencoders for collaborative filtering [9] further improve upon this by introducing a probabilistic latent space, which acts as a regularizer and has been shown to improve ranking performance. Collaborative Denoising Autoencoders (CDAE) [10] add input corruption during training to encourage more robust latent representations.

Content-based filtering [11] takes a complementary approach, representing items by their attributes and recommending items that are similar to those a user has previously engaged with. While content-based methods tend to underperform collaborative filtering on accuracy metrics, they are known to produce more diverse and novel recommendations [12], which is particularly relevant in domains with rich item metadata such as board games.

Hybrid approaches that combine collaborative and content-based signals have been explored as a way to balance accuracy with diversity. The work of Ion et al. [1] is the most directly relevant to this paper, as it applies and evaluates all of these paradigms in the specific domain of board games, a domain that has received comparatively little attention in the recommender systems literature despite its rich data availability.

Reproducibility in recommender systems research has received growing attention in recent years. Studies have repeatedly found that results from published papers are difficult to reproduce due to missing implementation details, unavailable datasets, and sensitivity to preprocessing choices. Our work contributes to this line of inquiry by attempting a faithful replication of Ion et al. [1] and documenting the sources of divergence we encounter.

3. Data

3.1 Original Dataset

The original paper assembled its dataset by scraping BoardGameGeek (BGG) via its public API [1]. BGG is the largest online community for board game enthusiasts, hosting user ratings, reviews, and rich structured metadata for tens of thousands of games. The original dataset comprised more than 13 million ratings on 80,474 games from 249,186 users. Approximately 85% of users had rated more than 50 games. Ratings are given on a scale from 0 to 10 and are normalized per user. The authors did not release this dataset, and the scraper used to construct it is not publicly available, making direct replication of the data impossible.

3.2 Dataset

Since the original dataset is unavailable, we use a publicly available Kaggle dataset scraped from BGG [2]. This dataset was selected because it is the closest available approximation to the original: it contains all the fields required for our evaluation, including user ratings, game mechanics (as binary features), game categories, playing time, minimum players, minimum age, and game weight. These attributes are essential not only for the content-based and hybrid recommenders but also for computing the Diversity and Novelty metrics used in evaluation. Other available BGG snapshots on Kaggle lack one or more of these fields and are therefore unsuitable for a full replication.

In its raw form, this dataset contains 18,942,152 ratings across 21,925 games from 411,374 users, making it larger than the original in terms of users and ratings but smaller in terms of the number of games covered. Following the original paper, we filter the dataset to retain only users with a minimum of 200 ratings, consistent with the paper's stated criterion for drawing useful conclusions. After filtering, our working dataset comprises 8,081,054 ratings across 21,919 games from 20,310 users. Table 1 summarises the comparison between the two datasets.

Table 1: Dataset comparison

Dataset	Users	Games	Ratings
Original (Ion et al. [1])	249,186	80,474	13,000,000+
Ours (raw)	411,374	21,925	18,942,152
Ours (filtered)	20,310	21,919	8,081,054

Ratings are normalized per user by subtracting each user's mean training rating, consistent with the paper's description.

3.3 Train/Test Split

Following the original paper, we perform a per-user random split of each user's ratings into 80% training data and 20% test data. This yields a training set of 6,477,630 ratings and a test set of 1,629,577 ratings. For evaluation, we define a relevant item as one with a held-out test rating of 7.0 or above, consistent with the threshold used for liked items elsewhere in the paper.

4. Methods

This paper aims to replicate the baseline algorithm proposed in [1] and to extend the benchmark with machine learning algorithms, including a denoising autoencoder and a graph neural network approach. The model will strictly follow the hyperparameters defined in the original paper. Because the original data and source code are unavailable, we expect variance in our benchmark. We will also provide fine-tuning results for the base models from the original work, trained on the BGG dataset. All models are implemented independently, with no shared components, to ensure no correlation or association between models.

4.1 Baseline: popularity

We use a popularity-based recommender as a baseline. The recommender ranks all items by the number of times they appear in the training set and recommends the globally most popular unseen items to each user. In many domains, such as movies and board games, popularity serves as a strong baseline because popular items are liked more frequently by users. This baseline requires no user-specific learning and serves as a lower bound for more sophisticated approaches.

4.2 Neighborhood-Based Collaborative Filtering

We implement two neighborhood-based collaborative filtering methods using the Surprise library. Both models compute a similarity score using mean-adjusted cosine similarity and generate recommendations based on the $k=5$ nearest neighbors, returning the top 100 unseen items per user.

User-User filtering computes similarities across users using their full normalized ratings vectors. Item-Item filtering computes similarities across items but restricts the rating matrix to ratings ≥ 7.0 before computing similarities, following the original paper.

4.3 Matrix Factorization

We implement a standard matrix factorization model with no user or item bias terms. The model decomposes the user and item into low-dimensional embeddings and uses stochastic gradient descent to learn the interactions between users and items by optimizing the loss function. User and item embeddings are initialized from a normal distribution with mean 0 and standard deviation 0.1. Following the original paper, the latent dimensionality is set to 5, the number of training epochs is 20, and the weight decay is

1e-4. The model is trained using the Adam optimizer with a learning rate of 1e-3. Ratings are mean-centered per user by subtracting each user's average training rating. At inference, predicted scores are de-normalized by adding back the user mean before ranking, and items seen in training are excluded.

4.4 Autoencoder-based models

We implement several neural autoencoder models: a classic autoencoder, a denoising autoencoder, a variational autoencoder, and a Collaborative Denoising Autoencoder. These models learn compact representations of user-item interactions and use them to predict recommendations.

All autoencoder models share the following common setup. Missing ratings are filled with zero. The reconstruction loss is a masked mean squared error (MSE) computed only over observed entries. Ratings are mean-centered per user by subtracting each user's average training rating before being used as input. At inference, predicted normalized scores are de-normalized by adding back the user mean before ranking, and items seen in training are excluded.

The classic autoencoder, denoising autoencoder, and variational autoencoder are each implemented in both a user-based and an item-based formulation. In the user-based formulation, each input vector represents one user's normalized ratings over all items, giving an input dimension of 21,919. In the item-based formulation, each input vector represents one item's normalized ratings across all users, giving an input dimension of 20,310. The architectures are identical between the two formulations - the only difference is the transposed input matrix and the corresponding change in input and output dimension.

4.4.1 Classic Autoencoder

The architecture consists of a single hidden layer with 1,024 nodes, a ReLU activation function, and a linear output layer. The model is trained with the Adadelta optimizer, a weight decay of 1e-2, a batch size of 128, and 50 epochs, consistent with Ion et al. [1].

4.4.2 Denoising Autoencoder

The denoising autoencoder follows the same architecture as the classic autoencoder but adds Gaussian noise to the input before encoding. Following Vincent et al. [12], which is the paper cited by Ion et al. [1] for the denoising approach, Gaussian noise with standard deviation 0.1 is added to observed entries only during training. Unobserved entries are left as zero and are not corrupted. All other hyperparameters are identical to the classic autoencoder. Both user-based and item-based variants are implemented.

4.4.3 Collaborative Denoising Autoencoder (CDAE)

The CDAE extends the denoising autoencoder by adding a learned identity-specific embedding to the hidden representation, following Wu et al. [10]. After encoding the corrupted input, a learned embedding vector unique to the user or item is added to the hidden state before the ReLU activation and decoding. This allows the model to condition its reconstruction on the specific user or item identity rather than learning a single shared denoising function across all inputs.

The corruption method used in the CDAE is Bernoulli dropout masking with corruption rate 0.1, following Wu et al., where each observed rating is independently zeroed out with probability 0.1. The architecture uses a hidden dimension of 1,024, ReLU activation, linear output, Adadelta optimizer, weight decay 0.01, batch size 128, and 100 training epochs. Both user-based and item-based variants are implemented. In the user-based variant a learned user embedding is added to the hidden layer; in the item-based variant a learned item embedding is added instead, with the input and output dimension changing from 21,919 to 20,310 accordingly.

4.4.4 Variational Autoencoder

The VAE encoder maps the input to a vector, from which a latent code is sampled during training using the reparameterization trick. The decoder maps from the latent space back to the rating space using a ReLU hidden layer followed by a sigmoid output. Following the original paper, the model uses binary cross-entropy (BCE) loss, a hidden dimension of 256, a latent dimension of 256, a batch size of 64, 50 epochs, and a weight decay of $1e-5$. Since BCE requires inputs in $[0, 1]$, min-max scaling is applied to the observed normalized ratings before training, computed from training entries only. The VAE loss is the sum of the BCE reconstruction term and KL divergence with weight 1. At inference, the posterior mean μ is used rather than a sampled latent vector. Both user-based and item-based variants are implemented, with the input and output dimension changing from 21,919 to 20,310 in the item-based variant.

4.4.5 Hyperparameter Tuning

We perform systematic hyperparameter tuning on the classic autoencoder in both user-based and item-based formulations to find configurations that improve over the paper's default settings. The loss function (masked MSE), activation (ReLU), and architecture (single hidden layer) are fixed across all tuning runs as specified in the original paper.

For the user-based autoencoder, the following values are searched: hidden dimension in $\{512, 1024, 2048\}$, weight decay in $\{0.001, 0.01, 0.1, 0.15, 0.2, 0.25, 0.3, 0.4\}$, batch size in $\{128, 256\}$, epochs in $\{50, 75, 100, 150\}$, and optimizer in $\{\text{Adadelata}, \text{Adam}(\text{lr}=1e-3)\}$. The best configuration found is hidden dimension 1,024, weight decay 0.2, batch size 256, 100 epochs, and Adadelata.

For the item-based autoencoder, the search covers hidden dimension in $\{512, 768, 1024, 1280, 1536, 2048\}$, weight decay in $\{0.0005, 0.001, 0.0025, 0.005, 0.01, 0.05, 0.1\}$, batch size in $\{128, 256\}$, and epochs in $\{50, 75, 100, 125\}$, with Adadelata fixed as the optimizer. The best configuration found is hidden dimension 1,024, weight decay 0.001, batch size 256, and 75 epochs.

4.5 Content-based

We implement two content-based recommenders using item features constructed from three sources: game mechanics (157 binary features), game categories (8 binary features), and three numeric attributes - game weight, minimum players, and manufacturing playtime. The numeric features are min-max scaled to $[0, 1]$ before use, giving a total feature matrix of 170 columns per item. Both models generate recommendations from items the user has liked in training, defined as ratings ≥ 7.0 .

4.5.1 kNN Content Filtering

We compute item-item euclidean distances from the 170-dimensional feature vectors and identify the top-10 most similar items to each game in the user's liked set. Neighbors are weighted using inverse-distance weighting where $w = 1 / (1 + d)$, so that closer neighbors contribute more to the final score. The recommendation score for each unseen item is the sum of weighted neighbor scores across all of the user's liked items. The top 100 unseen items by accumulated score are recommended.

4.5.2 IDF Filtering

The IDF recommender assigns higher weights to item features that appear less frequently across the dataset. For each user, a profile vector is constructed by taking the IDF-weighted sum of feature vectors of all items the user has liked. Unseen items are ranked by cosine similarity to this profile vector, and the top 100 items are recommended.

4.6 Graph-based

The GNN is implemented using a Light Graph Convolutional Network (LightGCN). It learns user-item interactions only from positive interactions, defined as ratings ≥ 7.0 , resulting in 3,873,046 positive training edges. Users and items are mapped into a shared index space, with item indices offset by the number of users to avoid collision. The bipartite graph is treated as undirected by including both directions of each edge. The model learns low-dimensional representations of users and items which are propagated through 3 light graph convolutional layers. Unlike the autoencoder models, the GNN does not operate on the full rating matrix - it only uses the graph structure of positive interactions and does not incorporate rating values beyond the binary liked/not-liked threshold. The model is trained using Bayesian Personalized Ranking (BPR) loss with L2 regularization ($\lambda = 1e-4$), an Adam optimizer with a learning rate of 0.001, and a batch size of 1,024. Training runs for 500 iterations. The implementation is adapted from AI Alchemy with modifications for our dataset.

4.7 hybrid model

The hybrid autoencoder takes as input the user's ratings on all items like a normal autoencoder and also takes as input content-based summary features of all the items that the user has liked (ratings ≥ 7.0). Content-based features include: TF-IDF of liked game category counts relative to all users, TF-IDF of liked game mechanics counts relative to all users, and average values of other game attributes: playing time, weight, minimum players, and minimum age. The input to the encoder is a concatenation of the user's normalized rating vector and this content summary vector. We combined the user-based classic autoencoder pipeline and the IDF recommender to create the hybrid autoencoder. The architecture uses 1,024 hidden nodes, 50 training epochs, a batch size of 128, and a regularization strength of 0.01, following Ion et al. [1]. This model is implemented in a user-based formulation only, as the content summary vector is defined per user based on their liked items.

5. Evaluation

We compared the recommendation metrics obtained by the original work and against the results obtained in our study in Table 2. The overall trend is that our replicated models performed worse on overall metrics, but showed similar ranking of algorithm performance as the original paper. The exception to this ranking is the hybrid autoencoder, which performed more like the content based algorithms than the collaborative filtering algorithms. All of the algorithms we trained had lower precision than the original paper. All of the algorithms we trained had higher novelty than the original paper.

We compared the recommendation metrics of user based autoencoder models with item based autoencoder models in Table 3. For the paper hyperparameter classic and denoising autoencoders, using item based models increased all metrics except for novelty and diversity. The opposite happened with the tuned classic autoencoder and the variational autoencoder, where all metrics decreased with item based models except novelty and diversity.

We compared the recommendation metrics of models with the original paper's provided hyperparameters vs our own tuned hyperparameters in Table 4. Tuning the user based classic autoencoder hyperparameters increased the recommendation metrics scores greatly for most metrics at the cost of novelty and diversity. Tuning the item based autoencoder had less of a dramatic impact but still increased most metrics by a substantial amount except for precision and recall at 100 items.

We compared two matrix factorization (MF) methods to the popularity baseline in Table 5. The two methods tested were traditional neural network (NN) MF and graph neural network (GNN) MF. We compared the baseline to highlight that only the GNN was able to outperform the popularity baseline on this data set. The GNN performed significantly better than the traditional NN MF method in all metrics.

We compared the optimal hyperparameters obtained in our hyperparameter tuning against the original paper’s tuned hyperparameters in Table 6 for the user-based classic autoencoder and Table 7 for the item-based classic autoencoder. The user-based autoencoder benefited from a much larger weight-decay, larger hidden dimension, batch size, and number of epochs. The item-based autoencoder benefited from a smaller weight decay with larger hidden dimension, batch size, and number of epochs.

Table 2: Comparison of recommendation metrics for each algorithm from the original paper to our replication. For the classic, denoising, and variational autoencoders, we chose to compare the user based version that had the same hyperparameters as specified in the original paper. Red color indicates that the replicated metric decreased relative to the original paper while green indicates that the replicated metric increased relative to the original paper.

Algorithm	User-User		Item-Item		Matrix Factorization		Classic Autoencoder		Denoising Autoencoder		Variational Autoencoder		KNN		IDF		Hybrid		Popularity Baseline	
	Orig.	Repl.	Orig.	Repl.	Orig.	Repl.	Orig.	Repl.	Orig.	Repl.	Orig.	Repl.	Orig.	Repl.	Orig.	Repl.	Orig.	Repl.	Orig.	Repl.
Precision at 5	0.016	0.003	0.398	0.002	0.689	0.002	0.691	0.176	0.697	0.143	0.692	0.057	0.069	0.047	0.115	0.039	0.696	0.025	0.548	0.358
Precision at 10	0.014	0.004	0.379	0.001	0.628	0.002	0.640	0.134	0.651	0.111	0.648	0.071	0.068	0.037	0.106	0.038	0.639	0.037	0.544	0.310
Precision at 20	0.012	0.005	0.303	0.004	0.551	0.002	0.574	0.114	0.582	0.106	0.594	0.072	0.065	0.029	0.100	0.035	0.567	0.040	0.506	0.155
Precision at 100	0.011	0.006	0.130	0.005	0.364	0.002	0.404	0.067	0.408	0.064	0.418	0.049	0.040	0.016	0.072	0.025	0.403	0.034	0.361	0.031
Recall at 5	0.000	0.000	0.011	0.000	0.019	0.000	0.019	0.021	0.019	0.017	0.019	0.007	0.002	0.006	0.003	0.005	0.019	0.002	0.015	0.042
Recall at 10	0.001	0.001	0.020	0.001	0.034	0.000	0.035	0.032	0.036	0.027	0.036	0.016	0.003	0.009	0.005	0.009	0.035	0.008	0.030	0.072
Recall at 20	0.001	0.003	0.032	0.002	0.059	0.001	0.062	0.053	0.063	0.050	0.064	0.033	0.006	0.014	0.010	0.017	0.061	0.017	0.055	0.072
Recall at 100	0.006	0.015	0.066	0.013	0.184	0.002	0.206	0.154	0.209	0.145	0.214	0.108	0.019	0.036	0.034	0.059	0.206	0.072	0.188	0.072
NDCG at 5	0.002	0.003	0.129	0.002	0.339	0.003	0.339	0.192	0.339	0.162	0.317	0.063	0.024	0.048	0.046	0.041	0.339	0.023	0.244	0.397
NDCG at 10	0.002	0.004	0.138	0.002	0.337	0.003	0.340	0.161	0.340	0.136	0.316	0.072	0.024	0.041	0.046	0.041	0.339	0.032	0.259	0.365
NDCG at 20	0.002	0.005	0.133	0.003	0.335	0.003	0.337	0.143	0.332	0.129	0.319	0.076	0.026	0.036	0.049	0.040	0.335	0.037	0.264	0.245
NDCG at 100	0.003	0.010	0.108	0.008	0.331	0.004	0.344	0.132	0.343	0.122	0.337	0.083	0.028	0.033	0.057	0.047	0.344	0.050	0.284	0.125
MAP	0.023	0.001	0.017	0.001	0.017	0.000	0.022	0.033	0.022	0.029	0.073	0.014	0.000	0.006	0.005	0.006	0.021	0.007	0.006	0.044
Diversity	0.067	0.063	0.068	0.060	0.087	0.068	0.085	0.098	0.086	0.108	0.084	0.108	0.029	0.054	0.110	0.099	0.085	0.107	0.083	0.101
Novelty	0.076	0.122	0.075	0.113	0.084	0.115	0.083	0.134	0.084	0.131	0.082	0.142	0.063	0.102	0.109	0.128	0.083	0.127	0.081	0.127

Table 3: Comparison of recommendation metrics for each autoencoder algorithm for which user-based vs item-based models were tested. Red color indicates that the item based model metric decreased relative to the user based metric while green indicates that the item based model metric increased relative to the user based metric.

Algorithm	Classic AE		Tuned Classic AE		Denoising AE CDAE		Denoising AE Gaussian Noise		Variational Autoencoder	
User or Item Based	User	Item	User	Item	User	Item	User	Item	User	Item
Precision at 5	0.176	0.233	0.290	0.274	0.126	0.255	0.143	0.230	0.057	0.012
Precision at 10	0.134	0.216	0.251	0.230	0.110	0.224	0.111	0.212	0.071	0.011
Precision at 20	0.114	0.187	0.205	0.188	0.099	0.190	0.106	0.180	0.072	0.011
Precision at 100	0.067	0.104	0.108	0.101	0.060	0.105	0.064	0.102	0.049	0.012
Recall at 5	0.021	0.027	0.033	0.032	0.015	0.029	0.017	0.026	0.007	0.001
Recall at 10	0.032	0.049	0.057	0.054	0.026	0.051	0.027	0.048	0.016	0.002
Recall at 20	0.053	0.084	0.092	0.087	0.047	0.085	0.050	0.080	0.033	0.004
Recall at 100	0.154	0.228	0.237	0.226	0.137	0.230	0.145	0.224	0.108	0.024
NDCG at 5	0.192	0.228	0.293	0.284	0.140	0.266	0.162	0.230	0.063	0.028
NDCG at 10	0.161	0.223	0.272	0.255	0.127	0.246	0.136	0.222	0.072	0.025
NDCG at 20	0.143	0.209	0.244	0.225	0.118	0.224	0.129	0.205	0.076	0.023
NDCG at 100	0.132	0.194	0.215	0.196	0.114	0.201	0.122	0.191	0.083	0.028
MAP	0.033	0.060	0.071	0.064	0.026	0.064	0.029	0.059	0.014	0.002
Diversity	0.098	0.083	0.078	0.098	0.105	0.083	0.108	0.082	0.108	0.114
Novelty	0.134	0.126	0.126	0.129	0.140	0.124	0.131	0.125	0.142	0.139

Table 4: Comparison of recommendation metrics for each autoencoder algorithm for which applying our own hyperparameter tuning was tested. Red color indicates that the tuned model metric decreased relative to the original paper hyperparameter model while green indicates that the tuned model metric increased relative to the original paper hyperparameter model.

Algorithm	User Based Classic AE		Item Based Classic AE	
	Paper	Tuned	Paper	Tuned
Precision at 5	0.176	0.290	0.233	0.274
Precision at 10	0.134	0.251	0.216	0.230
Precision at 20	0.114	0.205	0.187	0.188
Precision at 100	0.067	0.108	0.104	0.101
Recall at 5	0.021	0.033	0.027	0.032
Recall at 10	0.032	0.057	0.049	0.054
Recall at 20	0.053	0.092	0.084	0.087
Recall at 100	0.154	0.237	0.228	0.226
NDCG at 5	0.192	0.293	0.228	0.284
NDCG at 10	0.161	0.272	0.223	0.255
NDCG at 20	0.143	0.244	0.209	0.225
NDCG at 100	0.132	0.215	0.194	0.196
MAP	0.033	0.071	0.060	0.064
Diversity	0.098	0.078	0.083	0.098
Novelty	0.134	0.126	0.126	0.129

Table 5: Comparison of recommendation metrics for matrix factorization methods: traditional neural network vs graph neural network. Red color indicates that model metric is worse than baseline while green indicates that the model metric is better than baseline.

Algorithm	Neural Network Matrix Factorization	Graph Neural Network Matrix Factorization	Popularity Baseline
Precision at 5	0.001999	0.371374	0.358
Precision at 10	0.002152	0.317573	0.310
Precision at 20	0.002179	0.255844	0.155
Precision at 100	0.002064	0.128649	0.031
Recall at 5	0.000216	0.043388	0.042
Recall at 10	0.000451	0.073177	0.072
Recall at 20	0.000891	0.115843	0.072
Recall at 100	0.002064	0.282687	0.072
NDCG at 5	0.002573	0.391365	0.397
NDCG at 10	0.002744	0.360414	0.365
NDCG at 20	0.002863	0.320315	0.245
NDCG at 100	0.003967	0.273762	0.125
MAP	0.000237	0.096709	0.044
Diversity	0.068362	0.095209	0.101
Novelty	0.115324	0.122756	0.127

Table 6: Results of hyperparameter tuning for user-based classic autencoder.

Hyperparameter	Values Tested	Best Value
Architecture	1 hidden layer (input → hidden → output)	Same
Hidden Dimension	512, 1024, 2048	1024
Weight Decay (L2)	0.001, 0.01, 0.1, 0.15, 0.2, 0.25, 0.3, 0.4	0.2
Batch Size	128, 256	256
Epochs	50, 75, 100, 150	100
Optimizer	Adadelta, Adam (lr=1e-3)	Adadelta
Loss Function	Masked MSE (fixed, paper)	Same
Activation	ReLU (fixed, paper)	Same

Table 7: Results of hyperparameter tuning for item-based classic autencoder.

Hyperparameter	Values Tested	Best Value
Architecture	1 hidden layer (input → hidden → output)	Same
Hidden Dimension	512, 768, 1024, 1280, 1536, 2048	1024
Weight Decay (L2)	0.0005, 0.001, 0.0025, 0.005, 0.01, 0.05, 0.1	0.001
Batch Size	128, 256	256
Epochs	50, 75, 100, 125	75
Optimizer	Adadelta	Same
Loss Function	Masked MSE (fixed, paper)	Same
Activation	ReLU (fixed, paper)	Same

6. Extensions

We extended this paper in six ways: reimplementation of algorithm code, reimplementation of recommendation quality metric code, test of user-based vs item-based models, addition of new denoising autoencoder and graph neural network algorithms, hyperparameter tuning, and writing this report and sharing code and data. Each of these are described in more detail in Table 8.

Note: Please see the appendix for extension information relevant to CSCI 5123 Replication Project grading.

Table 8: Extensions of the original paper that this replication report delivers.

Extension	Details
Reimplementation of algorithm code	The original paper did not include any code to use for replication. We coded up each of the ten original algorithms by hand and shared our code.
Reimplementation of recommendation quality metric code	The original paper did not include any code to use for replication. We coded up each of the fifteen recommendation quality offline evaluation metrics by hand and shared our code.
Test of user-based vs item-based autoencoders	For the classic, denoising, and variational autoencoders, both user based and item based profiles were used to train autoencoder recommenders.
Addition of new denoising autoencoder and graph neural network algorithms	The original paper did not specify details of what type of denoising autoencoder was used. We tried using two methods: collaborative denoising Autoencoder (CDAE) and gaussian noise denoising autoencoder. Additionally, another matrix factorization method was applied using a graph neural network.
Hyperparameter tuning	The original paper provided the hyperparameters that worked best for their data set. We did our own independent tuning to test whether these hyperparameters are generalizable to another data set or if different hyperparameters would perform

	better.
Replication report	We wrote the results of our analyses into this report. We also include the code for all of our algorithms and evaluations.

7. Reproducibility Analysis and Discussion - Peer review

This section documents the assumptions we made during replication, the sources of divergence between our results and those of the original paper, and the lessons this study offers for reproducibility in recommender systems research.

7.1 Assumptions Made During Replication

Several implementation decisions were required because the original paper left key details unspecified. We document each assumption here along with the rationale for the choice made.

Relevance threshold. The paper does not define a numeric threshold for determining relevant items in evaluation. We use a threshold of 7.0 throughout, consistent with the threshold the paper applies in other contexts. For example, the Item-Item model restricts its similarity computation to ratings ≥ 7.0 , and the hybrid model builds content profiles from items rated above 7.0.

User-based vs. item-based autoencoders. The paper does not state whether the autoencoder operates over users or items. We implement both formulations and treat the user-based version as our primary replication, with the item-based version included as an alternative plausible interpretation.

Reconstruction loss masking. The paper does not specify whether the loss is computed over all entries or only observed ones. We compute MSE only over observed entries, so the model is not penalized for zero-filled missing ratings.

Decoder output activation. Not specified in the paper. We use a linear output for the classic and denoising autoencoders, and sigmoid for the VAE because BCE loss requires outputs in $[0, 1]$.

VAE latent dimension. The paper specifies 256 hidden nodes but does not separately specify the latent dimension. We set the latent dimension to 256 to match the hidden dimension.

Denoising corruption method. The paper specifies a denoising factor of 0.1 and cites Vincent et al. [12] but does not state the corruption type. We use additive Gaussian noise with standard deviation 0.1 applied to observed entries only, following Vincent et al.

Rating normalization formula. The paper states ratings are normalized per user but does not define the formula. We subtract each user's training-set mean rating without damping, which is the standard undamped mean-centering approach.

nDCG graded relevance. The paper states nDCG uses a graded relevance scale but does not specify the gain transformation. We use the raw held-out test rating (0-10) directly as the graded relevance value.

Item-Item neighborhood size. The paper does not specify a neighborhood size for Item-Item filtering. We use $k=5$, matching the value reported for User-User.

7.2 Dataset Mismatch and Its Impact

The most significant source of divergence between our results and those of the original paper is the dataset. The original paper scraped BoardGameGeek via a private scraper that is no longer available, producing a dataset of over 13 million ratings across 80,474 games from 249,186 users. Our dataset, sourced from Kaggle, covers 21,919 games from 20,310 users after filtering. The two datasets differ substantially in the number of games covered, as our dataset has roughly a quarter of the games, which affects the item space that models must learn over, the sparsity of the rating matrix, and the distribution of popular versus niche games.

The filtering criterion of 200 minimum ratings per user, taken directly from the paper, reduces our user base from 411,374 to 20,310, a much more aggressive reduction than what the original paper experienced. This means our filtered dataset represents a much smaller and potentially more active subset of BGG users, which may shift the distribution of ratings and affect how well popularity-based and neighborhood-based methods perform relative to neural approaches.

Despite these differences, the relative ordering of methods in our results is broadly consistent with the original paper. Autoencoders outperform neighborhood-based and content-based methods on ranking accuracy, the popularity baseline is a competitive benchmark, and content-based methods produce more diverse and novel recommendations. The quantitative values differ, which is expected and consistent with the dataset mismatch.

The difference in dataset also contributes to the observed difference in the sets of hyperparameters that optimize our algorithms' performance metrics. We suspect this difference in hyperparameters is what caused the hybrid method to perform worse than expected, as it needs to carefully balance the impact of collaborative filtering and content based features specific to the data set it is working with.

7.3 Scalability Challenges

The User-User and Item-Item models presented significant scalability challenges. Computing the full similarity matrix over 20,310 users or 21,919 items on a dataset of this size required approximately 30 hours of compute per model, making it impractical to run in a single session on Google Colab. We addressed this by implementing batched recommendation generation, splitting the computation across multiple sessions. This is worth noting for future replication efforts, as the original paper does not discuss the computational cost of these models on large datasets.

7.4 Underspecified Algorithmic Details

Beyond the assumptions listed in Section 7.1, several algorithmic details in the original paper are described at a level of abstraction that makes faithful replication difficult. The autoencoder section describes the architecture in terms of node count and training hyperparameters but leaves unspecified the masking strategy, the missing value encoding, the decoder activation, and the inference procedure. The denoising autoencoder section says the model adds distortion to its input but does not specify the corruption type. The hybrid model describes the content summary vector clearly but does not specify how the ratings vector and content vector are combined in the encoder input.

These gaps are representative of a broader challenge in recommender systems reproducibility. Even when a paper provides hyperparameter values, the evaluation outcomes are sensitive to preprocessing and implementation choices that are often treated as obvious but are in practice ambiguous. Our documentation of these decisions is intended to serve as a resource for future replication efforts.

7.5 Impact of Extensions

Our extended experiments provide additional insight beyond the original paper's findings. The item-based autoencoder variants consistently outperformed their user-based counterparts on our dataset, suggesting that the item-based formulation may be better suited to our dataset. Hyperparameter tuning produced meaningful improvements over the default settings from the paper, particularly for weight decay, indicating that the paper's reported hyperparameters may have been tuned for their specific dataset and do not transfer directly to ours. The CDAE achieved moderate performance, and the GNN achieved best results.

8. Conclusion

Recommendation systems research is notoriously difficult to replicate, and this replication paper shows in detail where this difficulty can arise from. The primary sources of difficulty were the lack of a published data set, the lack of publicly shared code, missing details on data preprocessing, and missing details on algorithms and metrics used for evaluation. Despite these challenges, we were able to replicate the main results of the paper in terms of the relative rank order performance of various algorithms with minor differences.

Our results reinforce the results in Ion et al. [1] which are that:

- Collaborative filtering based recommendation algorithms perform better than content based algorithms on recommendation ranking accuracy.
- Autoencoders based recommenders are able to achieve the best overall metric performance.
- The popularity baseline is a very strong benchmark in the board game domain.
- Content based methods provide recommendations that are more novel and diverse, by the definitions used in the original paper.
- The hybrid collaborative filtering and content based autoencoder does not provide any advantage over the purely collaborative filtering autoencoder.

Based on our results with this replication study, our main advice to recommender systems researchers is to

- Use a publicly available data set for public research, or make the data set that was used publicly available. Make sure the data set has a suitable data dictionary.
- Describe the data preprocessing steps in detail, and provide a flow chart showing what data is changed and what data is removed.
- Describe the algorithm implementation details well enough that someone else could recreate the exact same logic independently.
- Describe the evaluation metrics in enough detail that someone else could recreate the same results independently.
- Make the code used for performing the evaluation publicly available. This is primarily so that other researchers can more readily replicate and evaluate the research performed, and secondarily as a safety net to catch all details accidentally omitted in the written portion of the research publication.

References

- [1] Ion, M., Sacharidis, D., and Werthner, H. 2020. Designing a Recommender System for Board Games. In Proceedings of SAC '20. ACM, 1465-1467.
- [2] Threnjen. 2022. Board Games Database from BoardGameGeek. Kaggle. <https://www.kaggle.com/datasets/threnjen/board-games-database-from-boardgamegeek/data>
- [3] Griep, M. 2017. Hobby Games Market over \$1.4 Billion in 2016. ICv2.

- [4] Schwartz, B. 2009. The Paradox of Choice.
- [5] Sarwar, B., Karypis, G., Konstan, J., and Riedl, J. 2001. Item-based collaborative filtering recommendation algorithms. In Proceedings of WWW. ACM, 285-295.
- [6] Odic, A., Tkalcic, M., Tasic, J.F., and Kosir, A. 2012. Relevant context in a movie recommender system. ACM RecSys.
- [7] Koren, Y., Bell, R., and Volinsky, C. 2009. Matrix factorization techniques for recommender systems. Computer 42, 8.
- [8] Sedhain, S., Menon, A.K., Sanner, S., and Xie, L. 2015. AutoRec: Autoencoders meet collaborative filtering. In Proceedings of WWW. ACM, 111-112.
- [9] Liang, D., Krishnan, R.G., Hoffman, M.D., and Jebara, T. 2018. Variational Autoencoders for Collaborative Filtering. arXiv:1802.05814.
- [10] Wu, Y., DuBois, C., Zheng, A.X., and Ester, M. 2016. Collaborative denoising auto-encoders for top-n recommender systems. In Proceedings of WSDM. ACM, 153-162.
- [11] de Gemmis, M. et al. 2015. Semantics-Aware Content-Based Recommender Systems. In Recommender Systems Handbook. Springer, 119-159.
- [12] Castells, P., Vargas, S., and Wang, J. 2011. Novelty and diversity metrics for recommender systems.

Appendix: Extensions Details for Grading

We have implemented six extensions of the main report. Below is a table showing how each extension satisfies the requirements of the rubric.

Table A1: The six extensions we have completed for this project.

Extension	Details	Extension Type (as described on the Replication Assignment page)
1 - Reimplementation of algorithm code	The original paper did not include any code to use for replication. We coded up each of the ten original algorithms by hand.	Re-implementation or non-trivial coding / problem solving based around existing code.
2 - Reimplementation of recommendation quality metric code	The original paper did not include any code to use for replication. We coded up each of the fifteen recommendation quality offline evaluation metrics by hand.	Extending to new/multiple metrics. Credit for the “new/multiple” dataset/metrics/algorithms may be given in cases where the base paper already uses 3 or more datasets, metrics, algorithms.
3 - Test of user-based vs item-based autoencoders	For the classic, denoising, and variational autoencoders, both user based and item based profiles were used to train autoencoder recommenders.	Meaningful algorithmic tweaking or exploration of parameters

4 - Addition of new denoising autoencoder and graph neural network algorithms	The original paper did not specify details of what type of denoising autoencoder was used. We tried using two methods: collaborative denoising Autoencoder (CDAE) and gaussian noise denoising autoencoder. Additionally, another matrix factorization method was applied using a graph neural network.	Extending to new/multiple algorithms. Credit for the “new/multiple” dataset/metrics/algorithms may be given in cases where the base paper already uses 3 or more datasets, metrics, algorithms.
5 - Hyperparameter tuning	The original paper provided the hyperparameters that worked best for their data set. We did our own independent tuning to test whether these hyperparameters are generalizable to another data set or if different hyperparameters would perform better.	Meaningful algorithmic tweaking or exploration of parameters
6 - Replication report	We wrote the results of our analyses into this report. We also include the code for all of our algorithms and evaluations.	Writing the report in the form of a replication paper, rather than a simple summary report.